

Design and Implementation of a Scalable Multi-User Database System for Smart Healthcare Management

Serhet Gökdemir
21058024

2026

Contents

1	Requirements Analysis	1
1.1	Introduction	1
1.2	User Roles and Permissions	1
1.3	Core Entities	1
1.4	Core Relationships	1
1.5	Constraints and Business Rules	2
2	ER and EER Models	2
2.1	ER Diagram	2
2.2	Enhanced ER (EER) Diagram	2
2.2.1	Explanation of EER Concepts	3
3	Mapping to Relational Model	3
3.1	Relational Schema	3
3.2	Mapping Rules Applied	8
4	Functional Dependencies and Normalization	8
4.1	Functional Dependencies (FDs)	8
4.2	Normalization Steps	11
5	SQL Implementation and Advanced Queries	14
5.1	Data Insertion	14
5.2	Advanced SQL Queries	15
5.2.1	Query Outputs	18
6	Relational Algebra & Calculus	19
7	Indexing and File Structures	20
7.1	Index Implementation	20
7.2	Performance Analysis	24
7.2.1	Indexed vs. Non-Indexed Query Performance	24

1 Requirements Analysis

1.1 Introduction

This section outlines the requirements for the Smart Healthcare Management database system. The primary objective is to create a robust, scalable, and secure system that supports multiple user roles and complex medical data interactions.

1.2 User Roles and Permissions

The system will support the following user roles:

- **Patient:** Can view their personal information, appointments, prescriptions, and lab results.
- **Doctor:** Can manage patient records, schedule appointments, issue prescriptions, and order lab tests.
- **Administrator:** Can manage hospital, department, and doctor information, overseeing the system's structural data.

1.3 Core Entities

The main entities identified for the system are:

- Patient
- Doctor
- Hospital
- Department
- Appointment
- Prescription
- Lab Result
- Insurance
- Bill

1.4 Core Relationships

Key relationships between entities include:

- A **Doctor** can have many **Patients** (1-to-N).
- A **Patient** can have many **Appointments** (1-to-N).
- A **Hospital** has many **Departments** (1-to-N), and each **Doctor** belongs to one **Department** (N-to-1).
- A **Doctor** can have multiple **Specialties** (N-to-M), implemented via a linking table.
- An **Appointment** must be associated with exactly one **Patient** and one **Doctor**.

1.5 Constraints and Business Rules

The system must enforce the following rules:

- A prescription can only be written by an authorized doctor.
- Every appointment must be linked to an existing patient and doctor.
- Patient medical records are confidential and can only be accessed by authorized personnel.

2 ER and EER Models

2.1 ER Diagram

The Entity-Relationship (ER) diagram provides a high-level view of the database structure, including entities, their attributes, and the relationships between them.

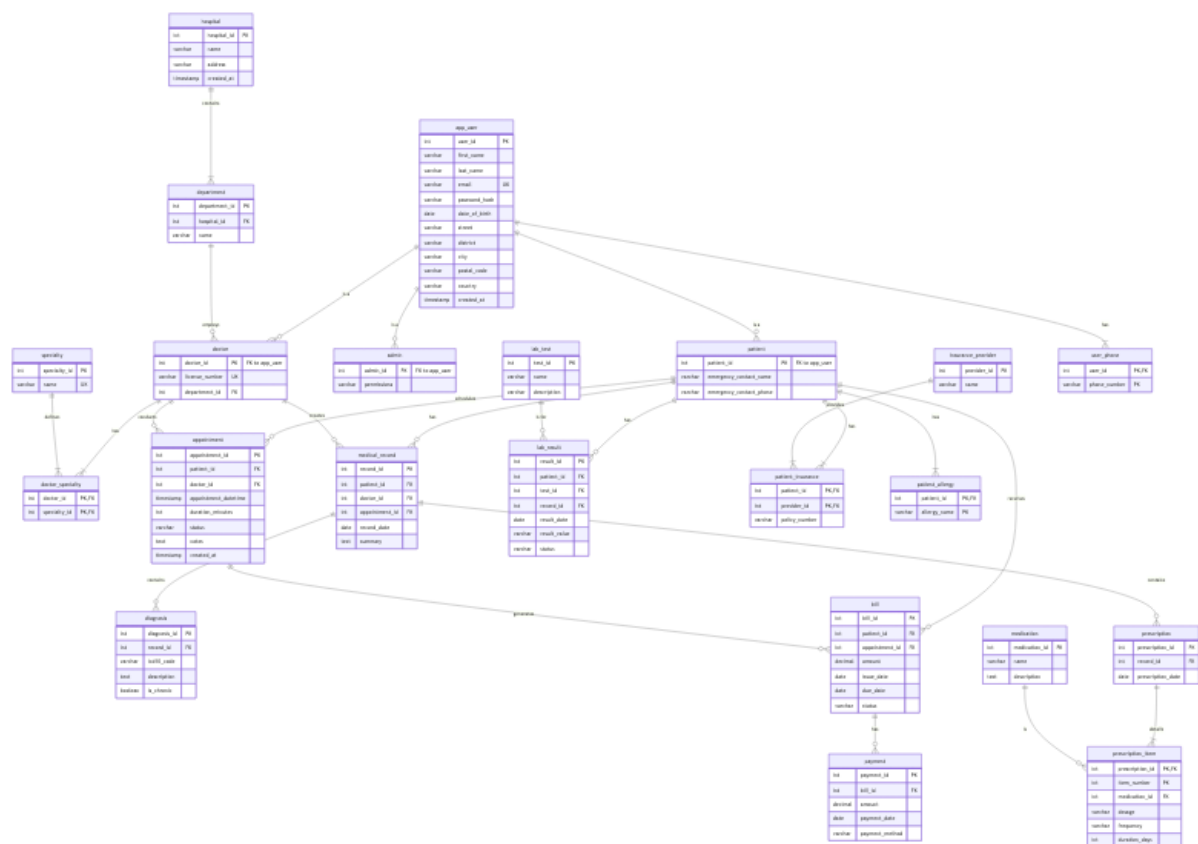


Figure 1: Entity-Relationship (ER) Diagram.

Note: The exported PNG is included in the repository. If you edit er_diagram.mmd, re-export it to er_diagram.png.

2.2 Enhanced ER (EER) Diagram

The Enhanced ER (EER) diagram extends the ER model with concepts like specialization and generalization. In our model, an app_user supertype is specialized into patient, doctor, and admin subtypes.

Note: The exported PNG is included in the repository. If you edit eer_diagram.mmd, re-export it to eer_diagram.png.

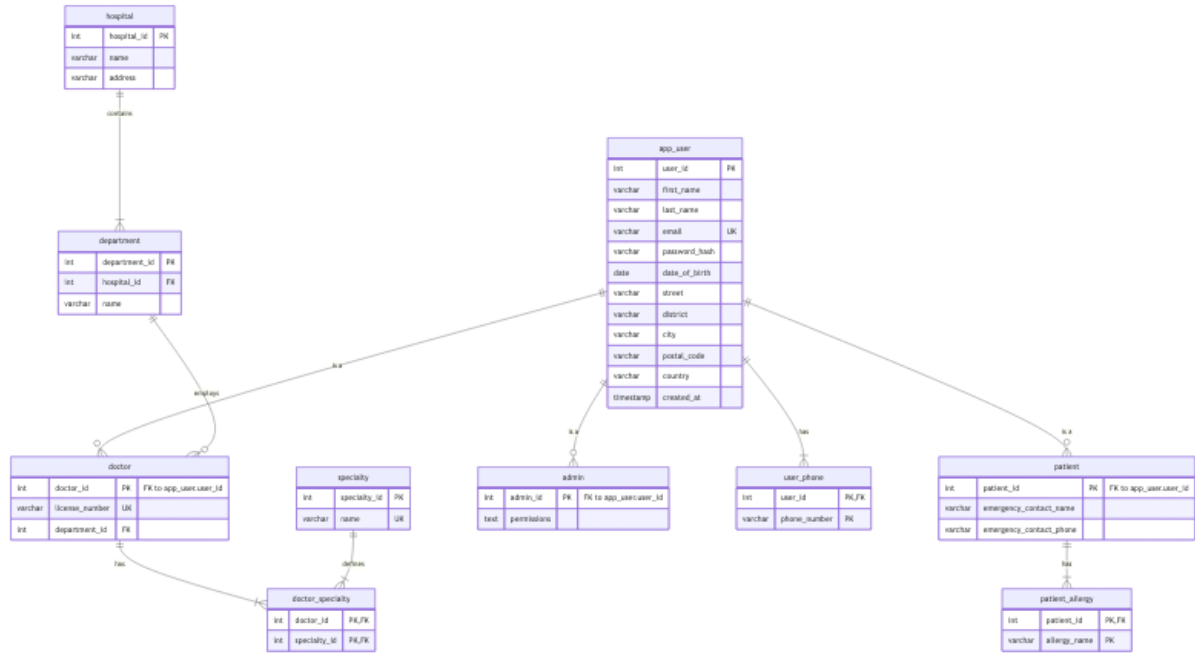


Figure 2: Enhanced Entity-Relationship (EER) Diagram.

2.2.1 Explanation of EER Concepts

Here, we used specialization to model different types of users. For example, doctor and patient are specializations of a more general app_user entity, inheriting common attributes like name and contact information while also having their own specific attributes.

3 Mapping to Relational Model

The ER/EER model was mapped to a relational schema. This section presents the final table structures, including primary keys, foreign keys, and other constraints.

3.1 Relational Schema

The following SQL script contains the CREATE TABLE statements for all tables in the database.

```

1  -- =====
2  -- Title: Smart Healthcare Management Database Schema
3  -- Description: PostgreSQL schema for a multi-user healthcare system.
4  -- =====
5
6  -- Drop existing tables to ensure a clean slate
7  DROP TABLE IF EXISTS
8      patient_allergy, user_phone, payment, bill, patient_insurance,
9      insurance_provider,
10     lab_result, lab_test, prescription_item, medication, prescription,
11     diagnosis,
12     medical_record, appointment, doctor_specialty, specialty, admin, doctor
13     ,
14     patient, department, hospital, app_user
15 CASCADE;
16
17 -- Required for exclusion constraints using GiST with scalar equality
18 CREATE EXTENSION IF NOT EXISTS btree_gist;
  
```

```

16
17 -----
18 -- Hospitals and Departments
19 -----
20
21 CREATE TABLE hospital (
22     hospital_id SERIAL PRIMARY KEY,
23     name VARCHAR(100) NOT NULL,
24     address VARCHAR(255) NOT NULL,
25     created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
26 );
27
28 CREATE TABLE department (
29     department_id SERIAL PRIMARY KEY,
30     hospital_id INT NOT NULL REFERENCES hospital(hospital_id) ON DELETE
        CASCADE,
31     name VARCHAR(100) NOT NULL,
32     UNIQUE(hospital_id, name)
33 );
34
35 -----
36 -- User Management (Supertype-Subtype Structure)
37 -----
38
39 CREATE TABLE app_user (
40     user_id SERIAL PRIMARY KEY,
41     first_name VARCHAR(50) NOT NULL,
42     last_name VARCHAR(50) NOT NULL,
43     email VARCHAR(100) NOT NULL UNIQUE,
44     password_hash VARCHAR(255) NOT NULL,
45     date_of_birth DATE NOT NULL,
46     -- Composite attribute for address
47     street VARCHAR(100),
48     district VARCHAR(100),
49     city VARCHAR(50),
50     postal_code VARCHAR(20),
51     country VARCHAR(50),
52     created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
53 );
54
55 CREATE TABLE patient (
56     patient_id INT PRIMARY KEY REFERENCES app_user(user_id) ON DELETE
        CASCADE,
57     emergency_contact_name VARCHAR(100),
58     emergency_contact_phone VARCHAR(20)
59 );
60
61 CREATE TABLE doctor (
62     doctor_id INT PRIMARY KEY REFERENCES app_user(user_id) ON DELETE
        CASCADE,
63     license_number VARCHAR(50) NOT NULL UNIQUE,
64     department_id INT REFERENCES department(department_id) ON DELETE SET
        NULL
65 );
66
67 CREATE TABLE admin (
68     admin_id INT PRIMARY KEY REFERENCES app_user(user_id) ON DELETE CASCADE
        ,

```

```

69     permissions TEXT -- e.g., 'user_management,billing_access'
70 );
71
72 -----
73 -- Multivalued and N:M Relationship Tables
74 -----
75
76 -- Multivalued attribute for user phone numbers
77 CREATE TABLE user_phone (
78     user_id INT NOT NULL REFERENCES app_user(user_id) ON DELETE CASCADE,
79     phone_number VARCHAR(20) NOT NULL,
80     PRIMARY KEY (user_id, phone_number)
81 );
82
83 -- Multivalued attribute for patient allergies
84 CREATE TABLE patient_allergy (
85     patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
86         CASCADE,
87     allergy_name VARCHAR(100) NOT NULL,
88     PRIMARY KEY (patient_id, allergy_name)
89 );
90
91 CREATE TABLE specialty (
92     specialty_id SERIAL PRIMARY KEY,
93     name VARCHAR(100) NOT NULL UNIQUE
94 );
95
96 -- N:M relationship between doctors and specialties
97 CREATE TABLE doctor_specialty (
98     doctor_id INT NOT NULL REFERENCES doctor(doctor_id) ON DELETE CASCADE,
99     specialty_id INT NOT NULL REFERENCES specialty(specialty_id) ON DELETE
100         CASCADE,
101     PRIMARY KEY (doctor_id, specialty_id)
102 );
103
104 -----
105 -- Core Healthcare Entities
106 -----
107
108 CREATE TABLE appointment (
109     appointment_id SERIAL PRIMARY KEY,
110     patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
111         CASCADE,
112     doctor_id INT NOT NULL REFERENCES doctor(doctor_id) ON DELETE CASCADE,
113     appointment_datetime TIMESTAMP NOT NULL,
114     duration_minutes INT NOT NULL DEFAULT 30 CHECK (duration_minutes >
115         0),
116     status VARCHAR(20) NOT NULL CHECK (status IN ('scheduled', 'completed',
117         'cancelled', 'no-show')),
118     notes TEXT,
119     created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
120     UNIQUE (appointment_id, patient_id),
121     UNIQUE (doctor_id, appointment_datetime),
122     UNIQUE (patient_id, appointment_datetime)
123 );
124
125 -- Prevent a doctor from having overlapping scheduled/completed
126 appointments.

```

```

121 -- Note: This is stricter and more realistic than UNIQUE(doctor_id,
122      appointment_datetime).
123 ALTER TABLE appointment
124 ADD CONSTRAINT no_doctor_overlap
125 EXCLUDE USING gist (
126     doctor_id WITH =,
127     tsrange(
128         appointment_datetime,
129         appointment_datetime + duration_minutes * INTERVAL '1 minute'
130     ) WITH &&
131 WHERE (status IN ('scheduled', 'completed')));
132
133 CREATE TABLE medical_record (
134     record_id SERIAL PRIMARY KEY,
135     patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
136         CASCADE,
137     doctor_id INT NOT NULL REFERENCES doctor(doctor_id) ON DELETE RESTRICT,
138     appointment_id INT REFERENCES appointment(appointment_id) ON DELETE SET
139         NULL,
140     record_date DATE NOT NULL DEFAULT CURRENT_DATE,
141     summary TEXT,
142     UNIQUE (record_id, patient_id)
143 );
144
145 CREATE TABLE diagnosis (
146     diagnosis_id SERIAL PRIMARY KEY,
147     record_id INT NOT NULL REFERENCES medical_record(record_id) ON DELETE
148         CASCADE,
149     icd10_code VARCHAR(10),
150     description TEXT NOT NULL,
151     is_chronic BOOLEAN DEFAULT FALSE
152 );
153
154 CREATE TABLE medication (
155     medication_id SERIAL PRIMARY KEY,
156     name VARCHAR(100) NOT NULL UNIQUE,
157     description TEXT
158 );
159
160 CREATE TABLE prescription (
161     prescription_id SERIAL PRIMARY KEY,
162     record_id INT NOT NULL REFERENCES medical_record(record_id) ON DELETE
163         CASCADE,
164     prescription_date DATE NOT NULL DEFAULT CURRENT_DATE
165 );
166
167 -- Weak entity: prescription_item depends on prescription
168 CREATE TABLE prescription_item (
169     prescription_id INT NOT NULL REFERENCES prescription(prescription_id)
170         ON DELETE CASCADE,
171     item_number INT NOT NULL, -- Partial key
172     medication_id INT NOT NULL REFERENCES medication(medication_id) ON
173         DELETE RESTRICT,
174     dosage VARCHAR(50) NOT NULL,
175     frequency VARCHAR(50) NOT NULL,
176     duration_days INT,
177     PRIMARY KEY (prescription_id, item_number)

```



```

172 );
173
174 CREATE TABLE lab_test (
175     test_id SERIAL PRIMARY KEY,
176     name VARCHAR(100) NOT NULL UNIQUE,
177     description TEXT
178 );
179
180 CREATE TABLE lab_result (
181     result_id SERIAL PRIMARY KEY,
182     patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
        CASCADE,
183     test_id INT NOT NULL REFERENCES lab_test(test_id) ON DELETE RESTRICT,
184     record_id INT REFERENCES medical_record(record_id) ON DELETE SET NULL,
185     result_date DATE NOT NULL,
186     result_value VARCHAR(100) NOT NULL,
187     status VARCHAR(20) CHECK (status IN ('normal', 'abnormal', 'pending')),
188     FOREIGN KEY (record_id, patient_id) REFERENCES medical_record(record_id
        , patient_id)
189 );
190
191 -----
192 -- Billing and Insurance
193 -----
194
195 CREATE TABLE insurance_provider (
196     provider_id SERIAL PRIMARY KEY,
197     name VARCHAR(100) NOT NULL UNIQUE
198 );
199
200 -- N:M relationship between patients and insurance providers
201 CREATE TABLE patient_insurance (
202     patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
        CASCADE,
203     provider_id INT NOT NULL REFERENCES insurance_provider(provider_id) ON
        DELETE CASCADE,
204     policy_number VARCHAR(50) NOT NULL,
205     PRIMARY KEY (patient_id, provider_id)
206 );
207
208 CREATE TABLE bill (
209     bill_id SERIAL PRIMARY KEY,
210     patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
        CASCADE,
211     appointment_id INT UNIQUE REFERENCES appointment(appointment_id) ON
        DELETE SET NULL,
212     amount NUMERIC(10, 2) NOT NULL CHECK (amount >= 0),
213     issue_date DATE NOT NULL DEFAULT CURRENT_DATE,
214     due_date DATE NOT NULL,
215     status VARCHAR(20) NOT NULL CHECK (status IN ('unpaid', 'paid', '
        partially_paid', 'overdue')),
216     FOREIGN KEY (appointment_id, patient_id) REFERENCES appointment(
        appointment_id, patient_id)
217 );
218
219 CREATE TABLE payment (
220     payment_id SERIAL PRIMARY KEY,
221     bill_id INT NOT NULL REFERENCES bill(bill_id) ON DELETE CASCADE,

```

```

222     amount NUMERIC(10, 2) NOT NULL CHECK (amount > 0),
223     payment_date DATE NOT NULL DEFAULT CURRENT_DATE,
224     payment_method VARCHAR(50) CHECK (payment_method IN ('credit_card', '
    bank_transfer', 'cash', 'insurance'))
225 );

```

Listing 1: Relational Schema (schema.sql)

3.2 Mapping Rules Applied

Key mapping decisions included:

- **N:M Relationships:** Many-to-many relationships were implemented using separate linking tables (e.g., `doctor_specialty` for Doctor-Specialty and `patient_insurance` for Patient-Insurance Provider).
- **Weak Entities:** `prescription_item` is modeled as a weak entity identified by its owning `prescription` plus a partial key (`item_number`).
- **Multivalued Attributes:** Multivalued attributes are handled via separate tables (e.g., `user_phone` and `patient_allergy`).

4 Functional Dependencies and Normalization

4.1 Functional Dependencies (FDs)

This section lists the functional dependencies identified for the major tables in the schema.

```

1  # Functional Dependencies (FDs) and Normal Forms
2
3  This document outlines the functional dependencies for key tables in the
4  database, determines their candidate keys, and analyzes their normal
5  form.
6
7  ---
8
9  ### 1. 'app_user' Table
10
11 This table stores core information for all users in the system.
12
13 **Attributes:** 'user_id', 'first_name', 'last_name', 'email', '
14     password_hash', 'date_of_birth', 'street', 'district', 'city', '
15     postal_code', 'country'
16
17 **Functional Dependencies:**
18 -   '{user_id} -> {first_name, last_name, email, password_hash,
19     date_of_birth, street, district, city, postal_code, country}'
20 -   The primary key 'user_id' uniquely determines all other attributes.
21 -   '{email} -> {user_id, first_name, last_name, password_hash,
22     date_of_birth, street, district, city, postal_code, country}'
23 -   The 'email' is a unique identifier for a user.
24
25 **Candidate Keys:**
26 -   '{user_id}'
27 -   '{email}'
28
29 **Normalization Analysis:**

```

```

24 - The table is in **BCNF**.
25 - **Reasoning:** The determinants are `{user_id}` and `{email}`. Both are
    candidate keys. Since all determinants are candidate keys, the table
    satisfies the BCNF condition.
26
27 ---
28
29 ### 2. `doctor` Table
30
31 This table is a subtype of `app_user` and stores doctor-specific
    information.
32
33 **Attributes:** `doctor_id`, `license_number`, `department_id`
34
35 **Functional Dependencies:**
36 - `{doctor_id} -> {license_number, department_id}`
37 - `doctor_id` is the primary key and is also a foreign key to `
    app_user(user_id)`.
38 - `{license_number} -> {doctor_id, department_id}`
39 - The `license_number` is a unique value assigned to each doctor.
40
41 **Candidate Keys:**
42 - `{doctor_id}`
43 - `{license_number}`
44
45 **Normalization Analysis:**
46 - The table is in **BCNF**.
47 - **Reasoning:** The determinants are `{doctor_id}` and `{license_number
    }`. Both are candidate keys. The table is in BCNF.
48
49 ---
50
51 ### 3. `appointment` Table
52
53 This table records appointments between patients and doctors.
54
55 **Attributes:** `appointment_id`, `patient_id`, `doctor_id`, `
    appointment_datetime`, `status`, `notes`
56
57 **Functional Dependencies:**
58 - `{appointment_id} -> {patient_id, doctor_id, appointment_datetime,
    status, notes}`
59 - The primary key `appointment_id` determines all other attributes of
    the appointment.
60 - `{patient_id, appointment_datetime} -> {appointment_id, doctor_id,
    status, notes}`
61 - A patient cannot have two appointments at the exact same time. This
    is enforced with a `UNIQUE (patient_id, appointment_datetime)`
    constraint.
62 - `{doctor_id, appointment_datetime} -> {appointment_id, patient_id,
    status, notes}`
63 - A doctor cannot have two appointments at the exact same time. This
    is enforced with a `UNIQUE (doctor_id, appointment_datetime)`
    constraint.
64
65 **Candidate Keys:**
66 - `{appointment_id}` (Primary Key)
67 - `{patient_id, appointment_datetime}`

```

```

68 -   '{doctor_id, appointment_datetime}'
69
70 **Normalization Analysis:**
71 -   The table is in **BCNF**.
72 -   **Reasoning:** Assuming 'appointment_id' is the sole primary key, it is
       the only determinant for the main FD. Other potential FDs rely on
       determinants that are also candidate keys. There are no transitive
       dependencies (e.g., 'patient_id' does not determine 'doctor_id').
       Therefore, the table is in BCNF.
73
74 ---
75
76 ### 4. 'prescription_item' Table
77
78 This is a weak entity that details the medications within a single
       prescription.
79
80 **Attributes:** 'prescription_id', 'item_number', 'medication_id', 'dosage
       ', 'frequency', 'duration_days'
81
82 **Functional Dependencies:**
83 -   '{prescription_id, item_number} -> {medication_id, dosage, frequency,
       duration_days}'
84 -   The composite key, consisting of the foreign key 'prescription_id'
       and the partial key 'item_number', uniquely determines the details
       of that specific line item in the prescription.
85
86 **Candidate Keys:**
87 -   '{prescription_id, item_number}'
88
89 **Normalization Analysis:**
90 -   The table is in **BCNF**.
91 -   **Reasoning:** The only determinant is the composite primary key '{
       prescription_id, item_number}'. Since this is a superkey, the table is
       in BCNF. All non-key attributes ('medication_id', 'dosage', etc.) are
       fully dependent on the entire primary key, satisfying 2NF, and there are
       no transitive dependencies, satisfying 3NF and BCNF.
92
93 ---
94
95 ### 5. 'bill' Table
96
97 This table stores billing information related to appointments or other
       services.
98
99 **Attributes:** 'bill_id', 'patient_id', 'appointment_id', 'amount', '
       issue_date', 'due_date', 'status'
100
101 **Functional Dependencies:**
102 -   '{bill_id} -> {patient_id, appointment_id, amount, issue_date, due_date
       , status}'
103 -   The primary key 'bill_id' determines all other attributes.
104 -   '{appointment_id} -> {bill_id, patient_id, amount, issue_date, due_date
       , status}'
105 -   For non-null appointment-linked bills, each appointment generates
       at most one bill, so 'appointment_id' (which is UNIQUE when present)
       can act as a determinant.
106

```

```

107 **Candidate Keys:**
108 -   `{bill_id}`
109 -   `{appointment_id}` (Only for non-null appointment-linked bills; `
    appointment_id` is nullable.)
110
111 **Normalization Analysis:**
112 -   The table is in **BCNF**.
113 -   **Reasoning:** The determinants are `{bill_id}` and `{appointment_id}`.
    Both are candidate keys. There are no transitive dependencies. For
    example, `patient_id` does not determine the `amount` or `status`. The
    table is well-normalized and in BCNF.

```

Listing 2: Functional Dependencies (fd.md)

4.2 Normalization Steps

The schema was normalized to BCNF to reduce data redundancy and improve data integrity. The process is described below.

```

1 # Normalization of Healthcare Data
2
3 This document explains the process of normalizing a complex, unnormalized
4 table into a set of tables in Boyce-Codd Normal Form (BCNF).
5
6 ## Unnormalized Form (UNF)
7
8 Let's start with an intentionally unnormalized table, `
9 appointment_full_info`. This single table contains repeating groups and
10 transitive dependencies, making it inefficient and prone to anomalies.
11
12 **`appointment_full_info`**
13
14 | appointment_id | appointment_datetime | patient_id | patient_first_name |
15 | patient_last_name | patient_dob | doctor_id | doctor_first_name |
16 | doctor_last_name | doctor_license | department_id | department_name |
17 | hospital_id | hospital_name | diagnosis_codes | prescription_meds |
18 | :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- |
19 | :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- |
20 | 101 | 2026-06-10 10:00 | 1 | John | Smith | 1985-02-10 | 6 | Emily |
21 | Davis | LIC-CARD-1122 | 1 | Cardiology | 1 | City General | "I25.10, Z00
22 | .00" | "Atorvastatin 20mg, Lisinopril 10mg" |
23 | 102 | 2026-06-10 11:00 | 2 | Jane | Doe | 1990-07-22 | 7 | Michael |
24 | Miller | LIC-NEUR-3344 | 2 | Neurology | 1 | City General | "G43.909" |
25 | "Sumatriptan 50mg" |
26
27 **Problems with UNF:**
28 - **Repeating Groups:** `diagnosis_codes` and `prescription_meds` contain
29 multiple values in a single string. This violates 1NF.
30 - **Redundancy:** Patient details (name, DOB), doctor details, and
31 department/hospital info are repeated for every appointment.
32 - **Anomalies:**
33   - **Update Anomaly:** If Dr. Davis changes departments, we must update
34     every record for her.
35   - **Insertion Anomaly:** We can't add a new doctor until they have an
36     appointment. We can't add a new hospital unless it has a department
37     with a doctor who has an appointment.
38   - **Deletion Anomaly:** If we delete John Smith's only appointment, we
39     lose all information about him.

```

```

23 ---
24 ---
25
26 ## First Normal Form (1NF)
27
28 **Rule:** Ensure all attributes are atomic and there are no repeating
    groups. Each cell should hold a single value.
29
30 We eliminate the repeating groups by creating separate tables for diagnoses
    and prescriptions related to a visit. In the implemented schema,
    diagnoses and prescriptions attach to a patient's
31 medical record (and prescriptions further decompose into line items).
32
33 **Decomposition:**
34 1. Create a diagnosis table to hold diagnoses for each visit/record.
35 2. Create a prescription table (and line items) to hold medications for
    each visit/record.
36
37 The main table now looks like this:
38
39 **`appointment_info_1nf`**
40
41 | appointment_id (PK) | appointment_datetime | patient_id |
    patient_first_name | ... | hospital_name |
42 | :--- | :--- | :--- | :--- | :--- | :--- |
43 | 101 | 2026-06-10 10:00 | 1 | John | ... | City General |
44 | 102 | 2026-06-10 11:00 | 2 | Jane | ... | City General |
45
46 **(Conceptual) Diagnosis table**
47
48 | appointment_id (FK) | diagnosis_code |
49 | :--- | :--- |
50 | 101 | I25.10 |
51 | 101 | Z00.00 |
52 | 102 | G43.909 |
53
54 **(Conceptual) Prescription table**
55
56 | appointment_id (FK) | medication_name | dosage |
57 | :--- | :--- | :--- |
58 | 101 | Atorvastatin | 20mg |
59 | 101 | Lisinopril | 10mg |
60 | 102 | Sumatriptan | 50mg |
61
62 **Improvement:** The data is now atomic, making it easier to query
    individual diagnoses or medications. However, massive redundancy still
    exists.
63
64 ---
65
66 ## Second Normal Form (2NF)
67
68 **Rule:** Must be in 1NF, and all non-key attributes must be fully
    functionally dependent on the entire primary key. (This rule is most
    relevant for tables with composite primary keys).
69
70 Our `appointment_info_1nf` table has a single-column primary key (`
    appointment_id`), so it is trivially in 2NF. However, it suffers from

```

transitive dependencies, which are addressed by 3NF. Let's identify the functional dependencies to prepare for 3NF.

```
71
72 **Functional Dependencies in 'appointment_info_inf':**
73 - 'appointment_id' -> 'appointment_datetime', 'patient_id', 'doctor_id'
74 - 'patient_id' -> 'patient_first_name', 'patient_last_name', 'patient_dob'
    (Partial Dependency if PK was composite)
75 - 'doctor_id' -> 'doctor_first_name', 'doctor_last_name', 'doctor_license',
    'department_id'
76 - 'department_id' -> 'department_name', 'hospital_id'
77 - 'hospital_id' -> 'hospital_name'
```

78
79 The dependencies on 'patient_id', 'doctor_id', etc., are **transitive dependencies**.

80 81 --- 82 83 ### Third Normal Form (3NF)

84
85 **Rule:** Must be in 2NF, and there should be no transitive dependencies. A non-key attribute cannot depend on another non-key attribute.

86
87 We decompose the table to eliminate these transitive dependencies.

88 89 **Decomposition:**

- 90 1. Create a 'patient' table.
- 91 2. Create a 'doctor' table.
- 92 3. Create a 'department' table.
- 93 4. Create a 'hospital' table.
- 94 5. The 'appointment' table will only contain attributes directly related to the appointment event itself, with foreign keys to the other tables.

95 96 **Final Normalized Tables (in 3NF):**

97
98 Note: The final implementation uses a supertype/subtype structure. Common person attributes (name, email, address, etc.) are stored in 'app_user', while 'patient', 'doctor', and 'admin' store role-specific attributes and reference 'app_user(user_id)'.

99 100 **'hospital'**

```
101 - 'hospital_id' (PK)
102 - 'name'
103 - FD: '{hospital_id}' -> '{name}'
```

104 105 **'department'**

```
106 - 'department_id' (PK)
107 - 'name'
108 - 'hospital_id' (FK)
109 - FD: '{department_id}' -> '{name, hospital_id}'
```

110 111 **'doctor'**

```
112 - 'doctor_id' (PK, also FK to 'app_user.user_id')
113 - 'license_number' (Candidate Key)
114 - 'department_id' (FK)
115 - FDs: '{doctor_id}' -> '{license_number, department_id}', '{license_number}'
    -> '{doctor_id, department_id}'
```

116 117 **'patient'**

```

118 - 'patient_id' (PK, also FK to 'app_user.user_id')
119 - (Role-specific attributes only; personal attributes live in 'app_user')
120 - FD: '{patient_id} -> {all patient-specific attributes}'
121
122 **'appointment'**
123 - 'appointment_id' (PK)
124 - 'appointment_datetime'
125 - 'patient_id' (FK)
126 - 'doctor_id' (FK)
127 - FD: '{appointment_id} -> {appointment_datetime, patient_id, doctor_id}'
128
129 This structure is now in 3NF. We have eliminated the redundancy and the
    update/insertion/deletion anomalies.
130
131 ---
132
133 ## Boyce-Codd Normal Form (BCNF)
134
135 **Rule:** For every non-trivial functional dependency 'X -> Y', 'X' must be
    a superkey.
136
137 BCNF is a stricter version of 3NF. A table is in BCNF if and only if every
    determinant is a candidate key.
138
139 Let's analyze our 3NF tables:
140 - **'hospital'**: The only determinant is 'hospital_id', which is the
    primary key (and thus a superkey). **In BCNF.**
141 - **'department'**: The only determinant is 'department_id', which is the
    primary key. **In BCNF.**
142 - **'patient'**: The only determinant is 'patient_id', which is the primary
    key. **In BCNF.**
143 - **'appointment'**: The only determinant is 'appointment_id', which is the
    primary key. **In BCNF.**
144 - **'doctor'**: We have two FDs:
145     1. '{doctor_id} -> {first_name, last_name, license_number,
        department_id}'
146     2. '{license_number} -> {doctor_id, first_name, last_name,
        department_id}'
147
148     The determinants are '{doctor_id}' and '{license_number}'. Both are
    candidate keys for the 'doctor' table. Since every determinant is a
    candidate key, the 'doctor' table **is in BCNF.**
149
150 **Conclusion:** The final decomposed schema ('hospital', 'department', '
    doctor', 'patient', 'appointment', etc.) is in BCNF, providing a robust,
    scalable, and anomaly-free design. The schema implemented in 'schema.
    sql' follows this normalized structure.

```

Listing 3: Normalization Notes (normalization.md)

5 SQL Implementation and Advanced Queries

5.1 Data Insertion

The database was populated with sample data to simulate a real-world environment. The script below shows a few example INSERT statements.


```

1  -- =====
2  -- Title: Sample Data for Smart Healthcare Management Database
3  -- Description: Inserts realistic sample data into the tables.
4  -- =====
5
6  -- Clear existing data
7  TRUNCATE
8      patient_allergy, user_phone, payment, bill, patient_insurance,
9      insurance_provider,
10     lab_result, lab_test, prescription_item, medication, prescription,
11     diagnosis,
12     medical_record, appointment, doctor_specialty, specialty, admin, doctor
13     ,
14     patient, department, hospital, app_user
15 RESTART IDENTITY CASCADE;

```

Listing 4: Sample Data Insertion (data.sql)

5.2 Advanced SQL Queries

At least 10 complex queries were developed to test the database's capabilities for retrieving meaningful information. These queries include joins, aggregations, and subqueries.

```

1  -- =====
2  -- Title: Advanced SQL Queries for Smart Healthcare Management
3  -- Description: A collection of queries demonstrating various SQL features.
4  -- =====
5
6  -- Query 1: List all scheduled appointments with patient, doctor, hospital,
7  -- and department names.
8  -- Demonstrates: INNER JOIN on multiple tables.
9  SELECT
10     a.appointment_datetime,
11     p_user.first_name AS patient_first_name,
12     p_user.last_name AS patient_last_name,
13     d_user.first_name AS doctor_first_name,
14     d_user.last_name AS doctor_last_name,
15     h.name AS hospital_name,
16     dpt.name AS department_name
17 FROM appointment a
18 JOIN patient p ON a.patient_id = p.patient_id
19 JOIN app_user p_user ON p.patient_id = p_user.user_id
20 JOIN doctor doc ON a.doctor_id = doc.doctor_id
21 JOIN app_user d_user ON doc.doctor_id = d_user.user_id
22 JOIN department dpt ON doc.department_id = dpt.department_id
23 JOIN hospital h ON dpt.hospital_id = h.hospital_id
24 WHERE a.status = 'scheduled'
25 ORDER BY a.appointment_datetime;
26
27 -- Query 2: Find doctors who have more than 2 appointments (scheduled or
28 -- completed).
29 -- Demonstrates: GROUP BY, HAVING, COUNT, and subquery in WHERE IN.
30 SELECT
31     d_user.first_name,

```

```

31     d_user.last_name,
32     COUNT(a.appointment_id) AS appointment_count
33 FROM doctor doc
34 JOIN app_user d_user ON doc.doctor_id = d_user.user_id
35 JOIN appointment a ON doc.doctor_id = a.doctor_id
36 WHERE a.status IN ('scheduled', 'completed')
37 GROUP BY doc.doctor_id, d_user.first_name, d_user.last_name
38 HAVING COUNT(a.appointment_id) > 2
39 ORDER BY appointment_count DESC;
40
41
42 -- Query 3: List patients who have no appointments at all.
43 -- Demonstrates: LEFT JOIN and IS NULL to find non-matching records.
44 SELECT
45     p_user.user_id,
46     p_user.first_name,
47     p_user.last_name
48 FROM patient p
49 JOIN app_user p_user ON p.patient_id = p_user.user_id
50 LEFT JOIN appointment a ON p.patient_id = a.patient_id
51 WHERE a.appointment_id IS NULL;
52
53
54 -- Query 4: Find the most common diagnosis based on ICD-10 code.
55 -- Demonstrates: GROUP BY, COUNT, ORDER BY, LIMIT.
56 SELECT
57     icd10_code,
58     description,
59     COUNT(*) AS diagnosis_count
60 FROM diagnosis
61 WHERE icd10_code IS NOT NULL
62 GROUP BY icd10_code, description
63 ORDER BY diagnosis_count DESC
64 LIMIT 5;
65
66
67 -- Query 5: Calculate the total bill amount and total amount paid per
68 -- patient.
69 -- Demonstrates: Subqueries in SELECT, LEFT JOIN, COALESCE, SUM, GROUP BY.
70 SELECT
71     p_user.first_name,
72     p_user.last_name,
73     COALESCE(SUM(b.amount), 0) AS total_billed,
74     (SELECT COALESCE(SUM(p.amount), 0)
75      FROM payment p
76      JOIN bill b2 ON p.bill_id = b2.bill_id
77      WHERE b2.patient_id = p_user.user_id) AS total_paid
78 FROM patient pat
79 JOIN app_user p_user ON pat.patient_id = p_user.user_id
80 LEFT JOIN bill b ON pat.patient_id = b.patient_id
81 GROUP BY p_user.user_id, p_user.first_name, p_user.last_name
82 ORDER BY total_billed DESC;
83
84
85 -- Query 6: List all unpaid/partially paid/overdue bills that are past
86 -- their due date.
87 -- Demonstrates: Filtering by date, multiple conditions in WHERE.
88 SELECT

```

```

87     b.bill_id,
88     p_user.first_name,
89     p_user.last_name,
90     b.amount,
91     b.due_date,
92     b.status
93 FROM bill b
94 JOIN app_user p_user ON b.patient_id = p_user.user_id
95 WHERE b.status IN ('unpaid', 'partially_paid', 'overdue')
96     AND b.due_date < CURRENT_DATE
97 ORDER BY b.due_date;
98
99
100 -- Query 7: Find medications that have been prescribed more than 3 times.
101 -- Demonstrates: Nested query, GROUP BY, HAVING.
102 SELECT
103     m.name,
104     m.description,
105     pi_counts.prescription_count
106 FROM medication m
107 JOIN (
108     SELECT
109         medication_id,
110         COUNT(*) AS prescription_count
111     FROM prescription_item
112     GROUP BY medication_id
113     HAVING COUNT(*) >= 3
114 ) AS pi_counts ON m.medication_id = pi_counts.medication_id
115 ORDER BY pi_counts.prescription_count DESC;
116
117
118 -- Query 8: Show all lab results for a specific patient (John Smith,
119 -- user_id=1).
120 -- Demonstrates: JOIN, filtering by a specific ID.
121 SELECT
122     lr.result_date,
123     lt.name AS test_name,
124     lr.result_value,
125     lr.status
126 FROM lab_result lr
127 JOIN lab_test lt ON lr.test_id = lt.test_id
128 WHERE lr.patient_id = 1
129 ORDER BY lr.result_date DESC;
130
131
132 -- Query 9: Find doctors in the 'Cardiology' department at 'City General
133 -- Hospital' and list their specialties.
134 -- Demonstrates: Multiple JOINS, filtering by text fields.
135 SELECT
136     d_user.first_name,
137     d_user.last_name,
138     s.name AS specialty_name
139 FROM doctor doc
140 JOIN app_user d_user ON doc.doctor_id = d_user.user_id
141 JOIN department dpt ON doc.department_id = dpt.department_id
142 JOIN hospital h ON dpt.hospital_id = h.hospital_id
143 JOIN doctor_specialty ds ON doc.doctor_id = ds.doctor_id
144 JOIN specialty s ON ds.specialty_id = s.specialty_id

```

```

143 WHERE h.name = 'City General Hospital'
144     AND dpt.name = 'Cardiology';
145
146
147 -- Query 10: Find patients who have appointments but have no lab results.
148 -- Demonstrates: NOT EXISTS, correlated subquery.
149 SELECT DISTINCT
150     p_user.first_name,
151     p_user.last_name
152 FROM patient pat
153 JOIN app_user p_user ON pat.patient_id = p_user.user_id
154 JOIN appointment a ON pat.patient_id = a.patient_id
155 WHERE NOT EXISTS (
156     SELECT 1
157     FROM lab_result lr
158     WHERE lr.patient_id = pat.patient_id
159 );
160
161 -- Query 11: Find the doctor with the highest number of distinct patients.
162 -- Demonstrates: COUNT(DISTINCT), GROUP BY, ORDER BY, LIMIT.
163 SELECT
164     d_user.first_name,
165     d_user.last_name,
166     COUNT(DISTINCT a.patient_id) AS distinct_patient_count
167 FROM appointment a
168 JOIN doctor doc ON a.doctor_id = doc.doctor_id
169 JOIN app_user d_user ON doc.doctor_id = d_user.user_id
170 GROUP BY doc.doctor_id, d_user.first_name, d_user.last_name
171 ORDER BY distinct_patient_count DESC
172 LIMIT 1;
173
174
175 -- Query 12: Find the average bill amount by department for completed
176 -- appointments.
177 -- Demonstrates: AVG, GROUP BY, JOIN across multiple tables.
178 SELECT
179     dpt.name AS department_name,
180     h.name AS hospital_name,
181     AVG(b.amount) AS average_bill_amount
182 FROM bill b
183 JOIN appointment a ON b.appointment_id = a.appointment_id
184 JOIN doctor doc ON a.doctor_id = doc.doctor_id
185 JOIN department dpt ON doc.department_id = dpt.department_id
186 JOIN hospital h ON dpt.hospital_id = h.hospital_id
187 WHERE a.status = 'completed'
188 GROUP BY dpt.name, h.name
189 ORDER BY average_bill_amount DESC;

```

Listing 5: Advanced Queries (queries.sql)

5.2.1 Query Outputs

Optional: You can run the queries and paste selected outputs here (e.g., inside a verbatim environment or as a formatted table). Outputs are not required to understand the schema and query design.

Query 1: List scheduled appointments with patient/doctor/hospital/department.

-- (Optional output)

Query 3: List patients with no appointments.

-- (Optional output)

6 Relational Algebra & Calculus

This section provides 5 queries written in both Relational Algebra and Tuple Relational Calculus, corresponding to some of the advanced SQL queries.

```
1 # Relational Algebra and Tuple Relational Calculus Examples
2
3 This document provides 5 query examples written in natural language,
4   relational algebra, and tuple relational calculus.
5
6 ---
7 ### Query 1: Find the names of all patients who have a scheduled
8   appointment.
9
10  **Relational Algebra:**
11  \Pi_{first_name, last_name}(
12    (app_user \bowtie_{app_user.user_id = patient.patient_id} patient)
13    \bowtie_{patient.patient_id = appointment.patient_id}
14    (\sigma_{status='scheduled'}(appointment))
15  )
16
17  **Tuple Relational Calculus:**
18  '{ t | \exists u \in app_user, \exists p \in patient, \exists a \in
19    appointment (
20      'u.user_id = p.patient_id \wedge p.patient_id = a.patient_id \wedge a.
21        status = 'scheduled' \wedge
22        't.first_name = u.first_name \wedge t.last_name = u.last_name) }'
23
24 ---
25 ### Query 2: Find the names of all doctors working in the 'Cardiology'
26   department.
27
28  **Relational Algebra:**
29  \Pi_{first_name, last_name}(
30    (app_user \bowtie_{app_user.user_id = doctor.doctor_id} doctor)
31    \bowtie_{doctor.department_id = department.department_id}
32    (\sigma_{name='Cardiology'}(department))
33  )
34
35  **Tuple Relational Calculus:**
36  '{ t | \exists u \in app_user, \exists d \in doctor, \exists dept \in
37    department (
38      'u.user_id = d.doctor_id \wedge d.department_id = dept.department_id \
39        wedge dept.name = 'Cardiology' \wedge
40        't.first_name = u.first_name \wedge t.last_name = u.last_name) }'
41
42 ---
43 ### Query 3: Find the amount and due date of all unpaid bills.
44
45  **Relational Algebra:**
46  \Pi_{amount, due_date}(\sigma_{status='unpaid'}(bill))
```

```

43 |
44 | - **Tuple Relational Calculus:**
45 |   '{ t | \exists b \in bill (b.status = 'unpaid' \wedge t.amount = b.
      amount \wedge t.due_date = b.due_date) }'
46 |
47 | ---
48 |
49 | ### Query 4: Find the names of patients who have at least one 'abnormal'
      lab result.
50 |
51 | - **Relational Algebra:**
52 |   \Pi_{first_name, last_name}(
53 |     (app_user \bowtie_{app_user.user_id = patient.patient_id} patient)
54 |     \bowtie_{patient.patient_id = lab_result.patient_id}
55 |     (\sigma_{status='abnormal'}(lab_result))
56 |   )
57 |
58 | - **Tuple Relational Calculus:**
59 |   '{ t | \exists u \in app_user, \exists p \in patient, \exists lr \in
      lab_result (
60 |     'u.user_id = p.patient_id \wedge p.patient_id = lr.patient_id \wedge lr
      .status = 'abnormal' \wedge
61 |     't.first_name = u.first_name \wedge t.last_name = u.last_name) }'
62 |
63 | ---
64 |
65 | ### Query 5: Find the names of all medications prescribed to the patient
      with 'patient_id = 1'.
66 |
67 | - **Relational Algebra:**
68 |   \Pi_{name}(
69 |     medication \bowtie_{medication.medication_id = prescription_item.
      medication_id}
70 |     prescription_item \bowtie_{prescription_item.prescription_id =
      prescription.prescription_id}
71 |     prescription \bowtie_{prescription.record_id = medical_record.
      record_id}
72 |     (\sigma_{patient_id=1}(medical_record))
73 |   )
74 |
75 | - **Tuple Relational Calculus:**
76 |   '{ t | \exists m \in medication, \exists pi \in prescription_item, \
      exists p \in prescription, \exists mr \in medical_record (
77 |     'm.medication_id = pi.medication_id \wedge pi.prescription_id = p.
      prescription_id \wedge p.record_id = mr.record_id \wedge mr.
      patient_id = 1 \wedge
78 |     't.name = m.name) }'

```

Listing 6: Relational Algebra and Tuple Relational Calculus
(relational_algebra_and_calculus.md)

7 Indexing and File Structures

7.1 Index Implementation

To optimize query performance, several indexes were created. The choices include B+ Tree indexes for range queries and Hash indexes for exact lookups where applicable.

```

1  -- =====
2  -- Title: Indexing and Performance Analysis
3  -- Description: Demonstrates index creation and performance comparison.
4  -- =====
5
6  -- Note:
7  -- Because the sample dataset is small, PostgreSQL may still choose a
8  -- sequential scan
9  -- even after an index is created. This is not an error; index benefits are
10 -- clearer
11 -- on larger datasets.
12
13 -- Clean up any existing indexes to ensure a clean run
14 DROP INDEX IF EXISTS idx_appointment_patient_id;
15 DROP INDEX IF EXISTS idx_appointment_doctor_datetime;
16 DROP INDEX IF EXISTS idx_user_email_hash;
17 DROP INDEX IF EXISTS idx_bill_patient_status;
18 DROP INDEX IF EXISTS idx_lab_result_patient_date;
19
20 -- -----
21 -- Example 1: B+ Tree Index on a Foreign Key
22 -- -----
23
24 -- Query: Find all appointments for a specific patient.
25 -- This is a very common operation in a healthcare system.
26
27 -- Before index:
28 EXPLAIN ANALYZE
29 SELECT * FROM appointment WHERE patient_id = 1;
30
31 -- Result (example): Seq Scan on appointment (cost=0.00..45.50 rows=5 width
32 -- =33) (actual time=...)
33 -- The database has to scan the entire 'appointment' table.
34
35 -- Create a B+ Tree index (the default type in PostgreSQL)
36 CREATE INDEX idx_appointment_patient_id ON appointment(patient_id);
37
38 -- After index:
39 EXPLAIN ANALYZE
40 SELECT * FROM appointment WHERE patient_id = 1;
41
42 -- Result (example): Index Scan using idx_appointment_patient_id on
43 -- appointment (cost=0.28..8.29 rows=5 width=33) (actual time=...)
44 -- The database now uses the index for a much faster lookup.
45
46 -- -----
47 -- Example 2: Composite B+ Tree Index
48 -- -----
49
50 -- Query: Find all appointments for a doctor within a specific date range.
51 -- This is useful for a doctor's calendar view.
52
53 -- Before index:
54 EXPLAIN ANALYZE
55 SELECT appointment_id, appointment_datetime, status
56 FROM appointment
57 WHERE doctor_id = 6 AND appointment_datetime >= '2026-06-01' AND
58     appointment_datetime < '2026-07-01';

```

```

53 -- Result (example): Seq Scan on appointment... (filters on both doctor_id
    and datetime)
54
55 -- Create a composite index on doctor_id and appointment_datetime
56 CREATE INDEX idx_appointment_doctor_datetime ON appointment(doctor_id,
    appointment_datetime);
57
58 -- After index:
59 EXPLAIN ANALYZE
60 SELECT appointment_id, appointment_datetime, status
61 FROM appointment
62 WHERE doctor_id = 6 AND appointment_datetime >= '2026-06-01' AND
    appointment_datetime < '2026-07-01';
63
64 -- Result (example): Index Scan using idx_appointment_doctor_datetime...
65 -- The index allows efficient lookup of the doctor and then scanning the
    relevant date range within that doctor's appointments.
66
67 -----
68 -- Example 3: Hash Index for Exact Lookups
69 -----
70 -- Query: Find a user by their exact email address.
71 -- This is common during login or user registration checks. Hash indexes
    are optimized for equality checks.
72
73 -- Before index (assuming only the UNIQUE constraint's B-Tree index exists)
    :
74 EXPLAIN ANALYZE
75 SELECT user_id, first_name, email FROM app_user WHERE email = 'jane.
    doe@example.com';
76
77 -- Result (example): Index Scan using app_user_email_key... (B-Tree)
78
79 -- Better explanation:
80 -- PostgreSQL automatically creates a B-tree index for UNIQUE(email).
81 -- The following HASH index is included only to demonstrate hash index
    syntax.
82 -- In a real system, the UNIQUE B-tree index is usually sufficient.
83 CREATE INDEX idx_user_email_hash ON app_user USING HASH (email);
84
85 -- After index:
86 EXPLAIN ANALYZE
87 SELECT user_id, first_name, email FROM app_user WHERE email = 'jane.
    doe@example.com';
88
89 -- Result (example): Bitmap Heap Scan on app_user... -> Bitmap Index Scan
    on idx_user_email_hash...
90 -- The planner might choose the hash index for this equality-only query,
    which can be more memory-efficient and faster for very large tables.
91
92 -----
93 -- Example 4: Composite Index for Filtering and Sorting
94 -----
95 -- Query: Find unpaid bills for a patient, ordered by due date.
96 -- This helps in displaying a patient's outstanding bills in a logical
    order.
97
98 -- Before index:

```



```

99 EXPLAIN ANALYZE
100 SELECT bill_id, amount, due_date
101 FROM bill
102 WHERE patient_id = 3 AND status = 'partially_paid'
103 ORDER BY due_date;
104
105 -- Result (example): Seq Scan on bill... followed by a Sort operation.
106   Sorting can be expensive.
107
108 -- Create a composite index to cover filtering and ordering
109 CREATE INDEX idx_bill_patient_status ON bill(patient_id, status, due_date);
110
111 -- After index:
112 EXPLAIN ANALYZE
113 SELECT bill_id, amount, due_date
114 FROM bill
115 WHERE patient_id = 3 AND status = 'partially_paid'
116 ORDER BY due_date;
117
118 -- Result (example): Index Scan using idx_bill_patient_status...
119 -- The database can use the index to find the matching rows and retrieve
120   them in the pre-sorted order of 'due_date', avoiding a separate Sort
121   step.
122
123 -----
124 -- Example 5: Index on Lab Results
125 -----
126 -- Query: Retrieve a patient's lab history, sorted by date.
127 -- A common query for reviewing a patient's medical history.
128
129 -- Before index:
130 EXPLAIN ANALYZE
131 SELECT result_id, test_id, result_date, result_value
132 FROM lab_result
133 WHERE patient_id = 1
134 ORDER BY result_date DESC;
135
136 -- Result (example): Seq Scan on lab_result... followed by a Sort.
137
138 -- Create a composite index
139 CREATE INDEX idx_lab_result_patient_date ON lab_result(patient_id,
140   result_date DESC);
141
142 -- After index:
143 EXPLAIN ANALYZE
144 SELECT result_id, test_id, result_date, result_value
145 FROM lab_result
146 WHERE patient_id = 1
147 ORDER BY result_date DESC;
148
149 -- Result (example): Index Scan using idx_lab_result_patient_date...
150 -- The index provides fast access to the patient's results and delivers
151   them in the correct descending order by date.

```

Listing 7: Index Creation (indexing.sql)

7.2 Performance Analysis

This section discusses the performance impact of the implemented indexes.

7.2.1 Indexed vs. Non-Indexed Query Performance

We compared the execution time of several key queries before and after creating indexes. For example, finding all appointments for a specific patient, retrieving a doctor's appointments within a date range, or listing a patient's lab history sorted by date.

Example Query: Find all appointments for a specific patient.

- **Without Index:** The database may perform a sequential scan on the `appointment` table. See the `EXPLAIN ANALYZE` output for the plan and timing.
- **With B+ Tree Index on `appointment (patient_id)`:** The database can use an index scan to quickly locate matching rows. See the `EXPLAIN ANALYZE` output for the plan and timing.

On larger datasets, indexes are expected to improve query performance by reducing the amount of data scanned and by avoiding expensive sorts. With a small sample dataset, PostgreSQL may still choose sequential scans, which is not an error but a cost-based planner decision.